

The String-to-String Correction Problem.

Journal of the Association for Computing Machinery, Vol. 21, No. 1, January 1974, p. 168-173.

問題定義

String-to-string correction problem 是在討論兩個字串間的距離，這邊的距離指的是將一個字串改成另一個字串的最少編輯操作次數（the minimum cost sequence of edit operations）。作者參考 Morgan 曾提出一個找出與關鍵字差距 1 的錯誤輸入的技術，其中用到四種 editing operations，而這篇文章會介紹一個計算此距離的演算法，用到其中三種 editing operations：change、insert 及 delete。此演算法的速度與字串長度的乘積有比例關係。String-to-string correction problem 可應用在拼字檢查及找到兩個字串的最長共同序列。

作者從一些基礎定義開始介紹，包含描述字串、計算 cost 函數的記號等。

關於 Edit Distance：

- 假設 A 是有限字串
 - $A\langle i \rangle$ 是 A 的第 i 個字元
 - $A\langle i:j \rangle$ 是 A 的第 i 到第 j 個字元（包含 j ）
 - 當 $i > j$, $A\langle i:j \rangle = \Lambda$ （空字串）
 - $|A|$ 為 A 的長度
- Edit operation 可以寫成 $a \rightarrow b$ ，其中 $(a, b) \neq (\Lambda, \Lambda)$
 - 若 $A = \text{oat}$, $B = \text{obt}$ ，且 B 是由 A 以 $a \rightarrow b$ 的操作而來，可寫成 $A \Rightarrow B$ via $a \rightarrow b$
 - Change operation： $a \neq \Lambda$ 且 $b \neq \Lambda$
 - Insert operation： $a = \Lambda$ 且 $b \neq \Lambda$
 - Delete operation： $a \neq \Lambda$ 且 $b = \Lambda$
- 假設 S 是一序列的 Edit operations $s_1, s_2, \dots, s_m$ ，且存在 S -derivation $A_1, A_2, \dots, A_m$ 使得 $A = A_0$ 且 $B = A_m$ 且 $A_{i-1} \Rightarrow A_i$ via s_i , $1 \leq i \leq m$ ；若從 A 到 B 中存在 S -derivation，可以說 S takes A to B
- 假設 γ 為計算字元操作 cost 的函數，可以得出一個非負真數 $\gamma(a \rightarrow b)$
 - 若 γ 的參數為 S ，則計算每個 operation 的總和： $\gamma(S) = \sum_{i=1}^m \gamma(s_i)$
 - 若 S 為 0 項，則 $\gamma(S) = 0$
 - $\delta(A, B)$ 表示最小 cost： $\delta(A, B) = \min\{\gamma(S) | S \text{ takes } A \text{ to } B\}$
 - 另外三個使演算法不失一般性的假設
 - ◆ $\gamma(a \rightarrow b) = \delta(a, b)$
 - ◆ $\gamma(a \rightarrow a) = 0$
 - ◆ $\gamma(a \rightarrow b) + \gamma(b \rightarrow c) \geq \gamma(a, c)$

關於 Traces：

- Traces 指的是一個字串如何變成另一個字串，包含兩個性質：
 - 對每個由 A 到 B 的 T 來說， $\gamma(S) = \text{cost}(T)$
 - 對每個 S 來說，存在 T 使得 $\text{cost}(T) \leq \gamma(S)$
- 也就是說一個字串變成另一個字串有很多種 Sequence，但一定有一種 cost 最小的 Trace
- 例如這樣就是一個 Trace：A 字串的 a 和 d 被刪除，B 字串被新增 d 和 a，連線則是替換的部分。連線一定要對應到各一個字母，且不能互相交叉。

字串 A：~~a~~ b c c d e a ~~d~~
 字串 B：b c d c a d e a

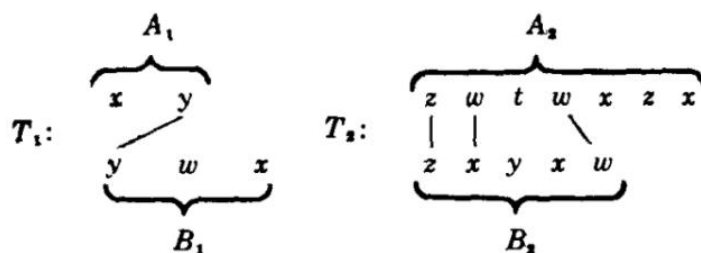
- T 的 cost 為 change、delete 及 insert 三種操作的 cost 的總和

$$\text{cost}(T) = \sum_{(i,j) \in T} \gamma(A\langle i \rangle \rightarrow B\langle j \rangle) + \sum_{i \in I} \gamma(A\langle i \rangle \rightarrow \Lambda) + \sum_{j \in J} \gamma(\Lambda \rightarrow B\langle j \rangle)$$
- 假設存在 (T_1, A, B) 、 (T_2, B, C) 及 (T, A, C) ，直接由 A 轉換成 C 的 cost 會比較小
 - $T = T_1 \circ T_2$
 - $\text{cost}(T_1 \circ T_2) \leq \text{cost}(T_1) + \text{cost}(T_2)$

解法敘述

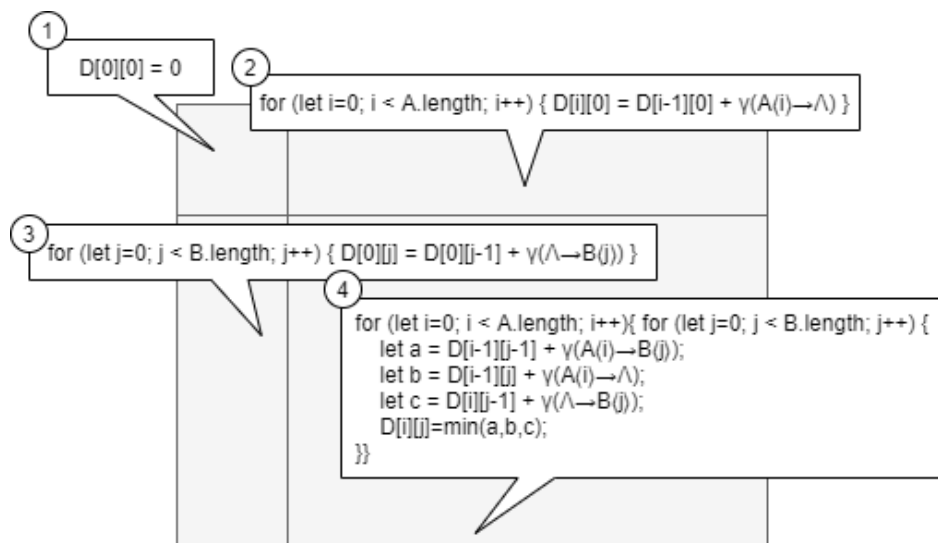
關於計算 Edit distance：

- 首先將字串分為互不影響的兩個區塊，如圖，當 $i \neq j$ ， A_i 和 B_j 間就不會有連線，只有區塊內才有連線。

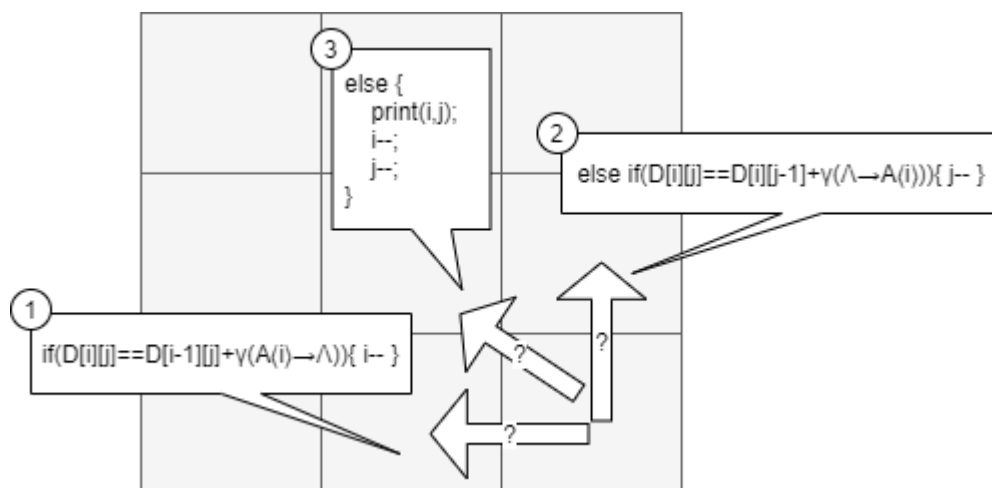


- 定義 $A(i)$ 及 $B(j)$ 為兩個字串到第 i 和第 j 個字元，則 $D(i,j)$ 表示 i 長度與 j 長度的字串間的最小 trace 的 cost。
- 由上圖可以知道， $A(i)$ 和 $B(j)$ 如果有連線，必定是相連（因為不能跨區，也沒有更右邊的字元）；因此可以只考慮 $A(i)$ 和 $B(j)$ 是屬於替換、新增還是刪除，算出 γ ，再加上左邊剩餘字串的 D ，最後再從這三者中找出最小值。
 - $D(i,j) = \min\{D(i-1,j-1) + \gamma(A\langle i \rangle \rightarrow B\langle j \rangle), D(i-1,j) + \gamma(A\langle i \rangle \rightarrow \Lambda), D(i,j-1) + \gamma(\Lambda \rightarrow B\langle j \rangle)\}$
- 若 i 或 j 為 0， γ 就是新增或刪除整條字串的 cost（前面字串的 D 加上 $\gamma(\Lambda \rightarrow B\langle j \rangle)$ 或 $\gamma(A\langle i \rangle \rightarrow \Lambda)$ ）；而當兩個字串都為空，cost 為 0。

- 由以上，我們可以列出計算 A 和 B 之間距離的演算法 X：
 - 首先宣告一個比 A 和 B 長度各加一為長寬的陣列 D，意義為 A 和 B 的子字串間的 D。
 - 依照上述前提，填入 D(0,0) 為 0，為下圖步驟一。
 - 當 i 或 j 為 0，D 為前面字串的 D 加上 $\gamma(\Lambda \rightarrow B(j))$ 或 $\gamma(A(i) \rightarrow \Lambda)$ ，為步驟二和三。
 - 計算剩餘格子，也就是新增刪除或替換的最小值，為步驟四。
 - 此算法的時間複雜度是 $1 + |A| + |B| + 4 \times |A| \times |B| = O(|A||B|)$ 。



- 此外也能藉由演算法 X 的結果，以 (i,j) 的形式印出 trace，為演算法 Y：
 - 由陣列 D 的最右下角開始，迭代以下步驟：
 - ◆ 如果當前格子是由左邊格子刪除一字元而來，則往左 (i--)。
 - ◆ 否則判斷是否由上面格子新增而來，是則往上 (j--)。
 - ◆ 否則往左上，且印出 (i,j) 。
 - 此演算法只印出往左上，即 A(i) 與 B(j) 相連，因而可以找出 trace。
 - 此演算法最多由右下角走到最頂又走到最左邊，為 $O(|A| + |B|)$ 。



關於 Longest Common Subsequences :

- 假設 $\rho(A, B)$ 是 Longest common subsequences 的長度， $T(A, B)$ 是相對應的 trace，且定義 γ 在新增或刪除時為 1，替換時為 2，相同時為 0，則：
 - $\text{cost}(T) = |A| + |B| - 2\rho(A, B)$ (將 A 全部刪除，再新增 B，但相同的地方被多算了 2 的 cost，所以要扣回來。)
 - 且 $\text{cost}(T) = \delta(A, B)$ ，故 $\rho(A, B) = (|A| + |B| - \delta(A, B))/2$ 。
 - 可以用演算法 X 算出 $\delta(A, B)$ 並得出 $\rho(A, B)$ ，並用演算法 Y 推出 LCS。

讀後心得

讀完了文章內容，對於演算法的論文有了新的認識。以前只看過有實驗設計、實驗過程、統計結果及後續討論的文章，並不了解演算法的論文要敘述哪些內容。我從這篇文章中看到，一個描述演算法的論文，必須先詳細定義各種前提，並以這些條件推導出後面的結果與方法。雖然也許演算法的結果計算出來是正確的，但是要描述為什麼必然會得到正確的結果很不容易。

從這篇文章中，可以看到如何用數學的方式表示一些概念。例如 Change Operation 是 $a \rightarrow b$ ，其中 $a \neq \Lambda$ 且 $b \neq \Lambda$ 。可以從這種描述裡面感受到「不失一般性」的特別之處。

在大二修 C++ 的時候就有寫過類似 LCS 的題目，也有因為遞迴直接跑不動過。雖然這篇文章沒有強調關於遞迴的部分，但它很簡要地讓我從演算法的結構中看到時間複雜度。可以更仔細地了解之前學過的一些東西，試著整理過，用自己的方式描述出來，對我來說是個很好的練習。